Lab 4: 多智能体系统

学习目标

通过本实验，你将：- 使用 CrewAI 构建多 Agent 协作系统 - 理解角色定义、任务分配、团队编排 - 实现一个完整的内容创作团队

环境准备

```
pip install crewai crewai-tools langchain-openai
```

实验任务

任务 1: 构建内容创作团队

```python
# crew_demo.py

from crewai import Agent, Task, Crew, Process
from langchain_openai import ChatOpenAI
from crewai_tools import SerperDevTool, WebsiteSearchTool

# 初始化 LLM
llm = ChatOpenAI(model="gpt-4", temperature=0.7)

# 初始化工具
search_tool = SerperDevTool()  # 需要设置 SERPER_API_KEY

# ============================================
# 1. 定义 Agent 角色
# ============================================

researcher = Agent(
    role=" 市场研究员",
    goal=" 深入调研{topic}的市场趋势、关键数据和最新动态",
    backstory="""  你是一位资深市场研究分析师，曾在麦肯锡和 BCG 工作多年。
    你擅长从海量信息中提炼关键洞察，总是用数据说话。
    你的研究报告以严谨和深度著称。""",
    verbose=True,
    allow_delegation=False,
    tools=[search_tool],
    llm=llm
)

writer = Agent(
    role=" 内容撰写专家",
```

```
    goal=" 基于研究资料撰写引人入胜、专业深度的文章",
    backstory="""  你是一位获奖财经作家，作品曾发表在《经济学人》《财富》等顶级刊物。
    你擅长将复杂的商业概念转化为生动易懂的故事。
    你的文章既有深度又有可读性。""",
    verbose=True,
    allow_delegation=False,
    llm=llm
)

editor = Agent(
    role=" 主编",
    goal=" 确保文章质量，进行最终审核和优化",
    backstory="""  你是一位有 20 年经验的资深主编，眼光独到、标准严格。
    你对事实准确性、逻辑连贯性和文字表达都有极高要求。
    经你之手的文章，质量都有显著提升。""",
    verbose=True,
    allow_delegation=True,
    llm=llm
)

seo_expert = Agent(
    role="SEO 优化专家",
    goal=" 优化文章的搜索引擎可见性",
    backstory="""  你是数字营销领域的 SEO 专家，帮助过数百篇文章获得搜索流量。
    你精通关键词研究、标题优化和元描述撰写。""",
    verbose=True,
    allow_delegation=False,
    llm=llm
)

# ==========================================
# 2. 定义任务
# ==========================================

research_task = Task(
    description="""
    调研"{topic}" 这个主题，收集以下信息:
    1. 市场现状和规模
    2. 主要玩家和竞争格局
    3. 最新趋势和重要事件
    4. 行业专家观点
    5. 关键数据和统计

    输出一份结构化的研究摘要，包含所有关键信息和数据来源。
    """,
    expected_output=" 一份 1000 字左右的研究摘要，包含关键数据和来源引用",
```

```
    agent=researcher
)

writing_task = Task(
    description="""
    基于研究摘要，撰写一篇关于"{topic}" 的深度文章：
    1. 引人入胜的开头
    2. 清晰的逻辑结构
    3. 数据支撑的观点
    4. 案例和故事
    5. 前瞻性的结论

    文章长度：2000-2500 字
    风格：专业但易读，适合商业人士阅读
    """,
    expected_output=" 一篇 2000-2500 字的专业文章",
    agent=writer,
    context=[research_task]  # 依赖研究任务的输出
)

editing_task = Task(
    description="""
    审核和优化文章：
    1. 检查事实准确性
    2. 优化文章结构
    3. 改进文字表达
    4. 确保观点有数据支撑
    5. 检查语法和用词

    如果发现重大问题，可以要求 Writer 修改。
    """,
    expected_output=" 审核通过的最终版文章，附带修改说明",
    agent=editor,
    context=[writing_task]
)

seo_task = Task(
    description="""
    为文章进行 SEO 优化：
    1. 优化标题（包含关键词，吸引点击）
    2. 撰写元描述（150 字以内）
    3. 建议小标题结构
    4. 提供 3-5 个相关关键词
    5. 建议内部/外部链接
    """,
    expected_output="SEO 优化建议清单和优化后的标题、元描述",
```

```python
    agent=seo_expert,
    context=[editing_task]
)

# ==========================================
# 3. 组建 Crew 并执行
# ==========================================

content_crew = Crew(
    agents=[researcher, writer, editor, seo_expert],
    tasks=[research_task, writing_task, editing_task, seo_task],
    process=Process.sequential,  # 顺序执行
    verbose=True
)

# 执行
def create_content(topic: str):
    print(f"\n{'='*60}")
    print(f" 开始创作主题: {topic}")
    print('='*60)

    result = content_crew.kickoff(inputs={"topic": topic})

    print(f"\n{'='*60}")
    print(" 创作完成!")
    print('='*60)
    print(result)

    return result

if __name__ == "__main__":
    # 测试
    create_content("2026 年 AI Agent 创业机会")
```

任务 2: 投资分析团队（并行模式）

```python
# investment_crew.py

from crewai import Agent, Task, Crew, Process

# 投资分析团队
bull_analyst = Agent(
    role=" 乐观派分析师",
    goal=" 找出{company}的投资亮点和增长潜力",
    backstory=" 你是一位成长股投资专家，擅长发现公司的增长动力。",
```

```
    verbose=True,
    llm=llm
)

bear_analyst = Agent(
    role=" 悲观派分析师",
    goal=" 找出{company}的风险因素和潜在问题",
    backstory=" 你是一位风险控制专家，擅长发现投资中的隐患。",
    verbose=True,
    llm=llm
)

quant_analyst = Agent(
    role=" 量化分析师",
    goal=" 用数据分析{company}的估值和财务健康度",
    backstory=" 你是 CFA 持证人，精通财务分析和估值模型。",
    verbose=True,
    llm=llm
)

chief_analyst = Agent(
    role=" 首席分析师",
    goal=" 综合各方观点，给出最终投资建议",
    backstory=" 你是首席分析师，有 20 年投研经验，擅长综合各方观点做出判断。",
    verbose=True,
    allow_delegation=True,
    llm=llm
)

# 任务定义
bull_task = Task(
    description=" 分析{company}的增长潜力、竞争优势和投资亮点",
    expected_output=" 看涨理由和目标价",
    agent=bull_analyst
)

bear_task = Task(
    description=" 分析{company}的风险因素、竞争威胁和潜在问题",
    expected_output=" 风险清单和最坏情景分析",
    agent=bear_analyst
)

quant_task = Task(
    description=" 分析{company}的财务数据、估值水平和同业对比",
    expected_output=" 估值分析和财务健康度评分",
    agent=quant_analyst
```

```
)

synthesis_task = Task(
    description="""
    综合乐观派、悲观派和量化分析师的观点，给出最终投资建议：
    1. 综合评分（1-10）
    2. 投资建议（买入/持有/卖出）
    3. 目标价和止损位
    4. 关键假设和风险提示
    """,
    expected_output=" 完整的投资建议报告",
    agent=chief_analyst,
    context=[bull_task, bear_task, quant_task]
)

# 组建 Crew
investment_crew = Crew(
    agents=[bull_analyst, bear_analyst, quant_analyst, chief_analyst],
    tasks=[bull_task, bear_task, quant_task, synthesis_task],
    process=Process.sequential,
    verbose=True
)

def analyze_investment(company: str):
    result = investment_crew.kickoff(inputs={"company": company})
    return result

if __name__ == "__main__":
    analyze_investment(" 英伟达 (NVIDIA)")
```

任务 3: 自定义工具集成

```
# custom_tools.py

from crewai_tools import BaseTool
from pydantic import BaseModel, Field

class StockPriceInput(BaseModel):
    ticker: str = Field(description=" 股票代码，如 TSLA，AAPL")

class StockPriceTool(BaseTool):
    name: str = " 股票价格查询"
    description: str = " 获取股票的当前价格和涨跌幅"
    args_schema: type[BaseModel] = StockPriceInput
```

```python
    def _run(self, ticker: str) -> str:
        # 实际应用中调用真实 API
        mock_data = {
            "TSLA": {"price": 248.50, "change": "+2.3%", "pe": 65.2},
            "AAPL": {"price": 185.20, "change": "-0.5%", "pe": 28.5},
            "NVDA": {"price": 890.00, "change": "+3.1%", "pe": 72.8},
        }
        ticker = ticker.upper()
        if ticker in mock_data:
            d = mock_data[ticker]
            return f"{ticker}: ${d['price']} ({d['change']}), P/E: {d['pe']}"
        return f" 未找到{ticker}的数据"

class NewsSearchInput(BaseModel):
    query: str = Field(description=" 搜索关键词")

class NewsSearchTool(BaseTool):
    name: str = " 新闻搜索"
    description: str = " 搜索最新的商业和财经新闻"
    args_schema: type[BaseModel] = NewsSearchInput

    def _run(self, query: str) -> str:
        # 模拟新闻搜索结果
        return f"""
关于"{query}" 的最新新闻:
1. [{query}相关公司发布最新财报，业绩超预期]
2. [行业分析师上调{query}相关股票评级]
3. [市场关注{query}领域的新技术突破]
"""

# 使用自定义工具
stock_tool = StockPriceTool()
news_tool = NewsSearchTool()

analyst_with_tools = Agent(
    role=" 研究分析师",
    goal=" 使用工具获取数据，进行深度分析",
    backstory=" 你是一位数据驱动的分析师",
    tools=[stock_tool, news_tool],
    verbose=True,
    llm=llm
)
```

提交要求

1. 完整的 `crew_demo.py` 代码
2. 运行日志截图（展示各 Agent 协作过程）
3. 最终生成的文章
4. 实验报告：
    - 各 Agent 的协作是否顺畅？
    - 最终输出质量如何？
    - 有什么改进建议？

进阶挑战

- 实现层级流程（Process.hierarchical）
- 添加人工审核节点
- 构建更复杂的协作模式

参考资源

- CrewAI 文档
- CrewAI GitHub